(https://blog.seeweb.it/autore/ad_seeweb/)
STAFF SEEWEB
(https://blog.seeweb.it/autore/ad_seeweb/)

Pubblicato in English Tech Guides (https://blog.seeweb.it/category/english-tech-guides/) 26 Settembre 2025

Getting Started with Kubeflow: GPU-Powered Machine Learning on Ubuntu

A hands-on guide to deploying Kubeflow with NVIDIA GPU support for high-performance model inference.

Indice dei contenuti

Machine learning in production requires more than just a trained model—you need reliable orchestration, repeatable deployments, seamless scaling and robust infrastructure. Kubeflow delivers exactly that by sitting on top of Kubernetes to provide portable, consistent MLOps that work everywhere: your laptop, on-premises servers, or in the cloud. With Seeweb’s reliable cloud-based GPU service, you can serve GPU-accelerated models without having to worry about strenuous set up effort and cost.

In this comprehensive guide, you’ll learn how to set up Kubeflow on Ubuntu using MicroK8s, enable NVIDIA GPU support, and serve a GPU-accelerated model with KServe backed by NVIDIA Triton Inference Server provided by Seeweb.

Why Choose Kubeflow and GPUs?

The Kubeflow Advantage

Kubeflow provides the essential building blocks for production MLOps:

- **Pipelines** for orchestrating complex, multi-step ML workflows
- **Notebooks** (Jupyter/VS Code) that run directly on cluster resources
- **KServe** for production-grade model serving with autoscaling, canary rollouts, and multi-framework support

The GPU Performance Boost

Modern deep learning models thrive on parallel processing, making GPUs essential for inference workloads. Here’s what you gain:

- **Lower latency**: Milliseconds instead of seconds
- **Higher throughput**: Process more predictions per second
- **Better resource utilization**: Maximize compute-intensive model performance

While this tutorial focuses on KServe + Triton for GPU-accelerated serving, you'll have access to the complete Kubeflow toolbox once your cluster is running.

What You'll Need

Before we begin, make sure you have:

- **Ubuntu 22.04+** (64-bit)
- **NVIDIA GPU** with recent drivers
- **8+ GB RAM** and **4+ CPU cores** (more for heavier models)
- **~20 GB free disk space** for container images

Pro Tip: If you're running on bare metal, prepare a small IP range (e.g., 10.64.140.43-10.64.140.49) for MetalLB, the load balancer used by MicroK8s in non-cloud environments.

Step 1: Set Up NVIDIA Drivers and Container Support

Before running GPU-accelerated Kubernetes workloads, we need to ensure your system can recognize and use the GPU properly. This involves installing the correct NVIDIA driver and enabling container runtime GPU support.

Install NVIDIA GPU Drivers

First, let's identify and install the recommended driver:

```
sudo apt update
ubuntu-drivers devices
# Install the recommended driver (example shows version 550)
sudo apt install -y nvidia-driver-550
sudo reboot
```

After rebooting, verify the installation:

```
nvidia-smi
```

You should see your GPU details, driver version, and CUDA information. If nothing appears, double-check your Secure Boot settings and driver versions.

NVIDIA Container Toolkit (Optional)

Note: MicroK8s uses containerd and its GPU addon automatically configures the NVIDIA runtime. Only install this toolkit manually if you want Docker containers on the host to access the GPU.

```
# Add NVIDIA's APT repository and install the toolkit
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
  sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt update
sudo apt install -y nvidia-container-toolkit

# Configure Docker (if you use it locally)
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# Quick Docker test
sudo docker run --rm --gpus all nvidia/cuda:12.2.0-base-ubuntu22.04 nvidia-smi
```

Step 2: Install Kubernetes with MicroK8s

Now we'll set up a Kubernetes cluster using MicroK8s—a lightweight, fully-featured Kubernetes environment that's perfect for both local development and small-scale production deployments.

Install MicroK8s and Configure kubectl

```
sudo snap install microk8s --classic
sudo usermod -a -G microk8s $USER
mkdir -p ~/.kube && chmod 700 ~/.kube

# Re-enter your shell to apply group membership
newgrp microk8s

# Optional: Use your host kubectl instead of microk8s kubectl
microk8s config > ~/.kube/config
```

Enable Essential Add-ons

Enable the required add-ons, including GPU support. **Remember to replace the MetalLB IP range with one appropriate for your network**:

```
microk8s enable dns hostpath-storage metallb:10.64.140.43-10.64.140.49 rbac gpu
```

Here's what each add-on provides:

- **dns**: Service discovery within the cluster
- **hostpath-storage**: Simple default StorageClass (ideal for single-node/development)
- **metallb**: External IP addresses for bare metal deployments
- **rbac**: Authentication and authorization primitives used by Kubeflow
- **gpu**: NVIDIA GPU Operator, device plugin, and container runtime configuration

Verify GPU Add-on Installation

Check that the GPU components are working correctly:

```
# The validator should report success
microk8s kubectl logs -n gpu-operator-resources \
  -l app=nvidia-operator-validator -c nvidia-operator-validator

# Optional: Run a quick CUDA test in the cluster
microk8s kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata: { name: cuda-vector-add }
spec:
  restartPolicy: OnFailure
  containers:
  - name: cuda
    image: k8s.gcr.io/cuda-vector-add:v0.1
    resources:
      limits:
        nvidia.com/gpu: 1
EOF

# Check the test results
microk8s kubectl logs cuda-vector-add
```

Wait for all pods to become ready:

```
microk8s kubectl get pods -A -w
```

Step 3: Deploy Kubeflow

With Kubernetes running, we'll deploy Charmed Kubeflow (CKF) using Juju. This installation includes all Kubeflow components: Pipelines, Notebooks, KServe, Istio, and authentication.

Install and Configure Juju

```
# Install Juju
sudo snap install juju --classic

# Bootstrap a controller on MicroK8s
juju bootstrap microk8s microk8s-localhost

# Create the Kubeflow model (must be named exactly 'kubeflow')
juju add-model kubeflow

# Deploy the latest stable CKF bundle
juju deploy kubeflow --trust

# Monitor the deployment progress
juju status --watch 5s
```

Configure Authentication and Access the Dashboard

Once all applications show as "active" in the juju status:

```
# Setup basic authentication credentials
juju config dex-auth static-username=admin
juju config dex-auth static-password=admin

# Get the dashboard IP address
microk8s kubectl -n kubeflow \
  get svc istio-ingressgateway-workload \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

Open the displayed IP address in your browser and log in using the credentials you just configured (admin/admin).

Important: Charmed Kubeflow includes its own Istio ingress gateway, so you don't need the MicroK8s ingress (NGINX) add-on for the standard installation.

Step 4: Deploy Your First GPU-Accelerated Model

Now for the exciting part! We'll deploy a sample TorchScript model for CIFAR-10 classification using KServe and NVIDIA Triton Inference Server. This example demonstrates how to request GPU resources and test the inference endpoint.

Create a User Workspace

In Kubeflow, each user should have an isolated Profile, which creates a Kubernetes namespace with proper RBAC and Istio settings.

Recommended approach: From the Kubeflow Dashboard, create a new Profile (e.g., "gputest").

Alternative approach: If you prefer creating a namespace manually, ensure it has the correct label:

```
kubectl create namespace gputest
kubectl label namespace gputest \
  serving.kubeflow.org/inferenceservice=enabled
```

Define the Inference Service

Create a file called torchscript-cifar.yaml:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: torchscript-cifar10
  namespace: gputest
spec:
  predictor:
    triton:
      # Use a Triton image tag supported by your cluster
      runtimeVersion: "20.10-py3"
      storageUri: "gs://kfserving-examples/models/torchscript"
      env:
        - name: OMP_NUM_THREADS
          value: "1"
      resources:
        limits:
          nvidia.com/gpu: 1
```

Deploy the service:

```
kubectl apply -f torchscript-cifar.yaml

# Monitor the deployment
kubectl get pods -n gputest -w
kubectl get inferenceservice torchscript-cifar10 -n gputest
```

Wait until the READY status shows “True”—your model is now live and ready for inference!

Test the Model Endpoint

First, gather the necessary connection details:

```
export INGRESS_HOST=$(kubectl -n kubeflow \
  get svc istio-ingressgateway-workload \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
export INGRESS_PORT=80

export SERVICE_HOSTNAME=$(kubectl -n gputest \
  get inferenceservice torchscript-cifar10 \
  -o jsonpath='{.status.url}' | cut -d "/" -f 3)
```

Now let’s test the model with sample data:

```
# Download the sample input data
curl -O
https://raw.githubusercontent.com/kserve/kserve/master/docs/samples/v1beta1/triton/torchscript/i

# Send a prediction request
MODEL_NAME=cifar10
curl -v -H "Host: ${SERVICE_HOSTNAME}" -H "Content-Type: application/json" \
  http://${INGRESS_HOST}:${INGRESS_PORT}/v2/models/${MODEL_NAME}/infer \
 -d @./input.json
```

You should receive a JSON response containing prediction scores for all 10 CIFAR-10 classes!

Step 5: Monitor GPU Usage and Performance

Real-time GPU Monitoring

Watch GPU utilization in real-time from your host system:

```
watch -n1 nvidia-smi
```

During inference, you should see a tritonserver process consuming GPU memory, with utilization spikes when processing requests.

Check Model Server Logs

View detailed logs from your inference service:

```
kubectl logs -n gputest \
  $(kubectl get pods -n gputest \
    -l serving.kserve.io/inferenceservice=torchscript-cifar10 \
    -o jsonpath='{.items[0].metadata.name}') \
  -c kserve-container
```

Load Testing

Test your model's performance under load with multiple concurrent requests:

```
# Send 10 parallel requests
for i in {1..10}; do
  curl -s -H "Host: ${SERVICE_HOSTNAME}" -H "Content-Type: application/json" \
    http://${INGRESS_HOST}:${INGRESS_PORT}/v2/models/${MODEL_NAME}/infer \
    -d @./input.json &
done; wait
```

Troubleshooting Common Issues

InferenceService Won't Start or Deploy

Issue: Service appears stuck or won't pull the model.

Solutions:

- Avoid deploying into control-plane namespaces (like kubeflow)
- Check for errors: kubectl describe inferenceservice <name> -n <namespace>
- For manually created namespaces, verify the label: serving.kubeflow.org/inferenceservice=enabled

GPU Not Available in Pods

Issue: Pods can't access the GPU.

Solutions:

- Confirm nvidia-smi works on the host
- Check GPU Operator status: kubectl get pods -n gpu-operator-resources
- Verify pod configuration: kubectl describe pod <pod> -n <namespace> and look for nvidia.com/gpu: 1 in limits

Can’t Access the Model Endpoint

Issue: Unable to reach the inference service.

Solutions:

- Verify MetalLB assigned an external IP: check istio-ingressgateway-workload service in the kubeflow namespace
- Test the gateway directly: `curl -I http://$INGRESS_HOST/`
- As a fallback, use port-forwarding: kubectl -n kubeflow port-forward svc/istio-ingressgateway-workload 8080:80 and access http://localhost:8080

Production Tips and Next Steps

Now that you have a working GPU-accelerated ML serving setup, consider these enhancements for production use:

User Management: Use Profiles for proper user isolation with correct RBAC and Istio configurations.

Auto-scaling: Leverage Knative’s ability to scale model servers to zero during idle periods and back up under load.

Model Versioning: Implement safe model rollouts using traffic splits and canary deployments.

Monitoring: Integrate Prometheus/Grafana for metrics and set up centralized log aggregation.

Security: Enable HTTPS and consider implementing Istio CNI for enhanced security.

Multi-tenancy: For high-end GPUs like A100/H100, explore MIG (Multi-Instance GPU) for partitioning GPUs across multiple tenants.

UI Management: Use the KServe Models Web App to create and manage InferenceService objects through a graphical interface.

Technical Appendix

Choosing the Right Triton Image Tag

The spec.predictor.triton.runtimeVersion field maps to a specific Triton container tag. Choose a tag that’s compatible with your driver and CUDA stack. This example uses 20.10-py3, which matches the upstream samples for broad compatibility.

GPU Resource Configuration

GPUs are extended resources in Kubernetes. It’s sufficient to set only the limits (e.g., nvidia.com/gpu: 1). If you also specify requests, they must exactly match the limit value.

Storage Considerations

The hostpath storage used in this guide works great for single-node development environments. For high-availability or multi-node clusters, consider using distributed storage backends.

Helpful Resources

- **Kubeflow**: https://www.kubeflow.org (https://www.kubeflow.org)
- **KServe Documentation**: https://kserve.github.io/website/ (https://kserve.github.io/website/)
- **Charmed Kubeflow**: https://charmed-kubeflow.io/docs (https://charmed-kubeflow.io/docs)
- **MicroK8s Documentation**: https://microk8s.io/docs (https://microk8s.io/docs)
- **NVIDIA Triton Inference Server**: https://developer.nvidia.com/triton-inference-server (https://developer.nvidia.com/triton-inference-server)

Articoli correlati

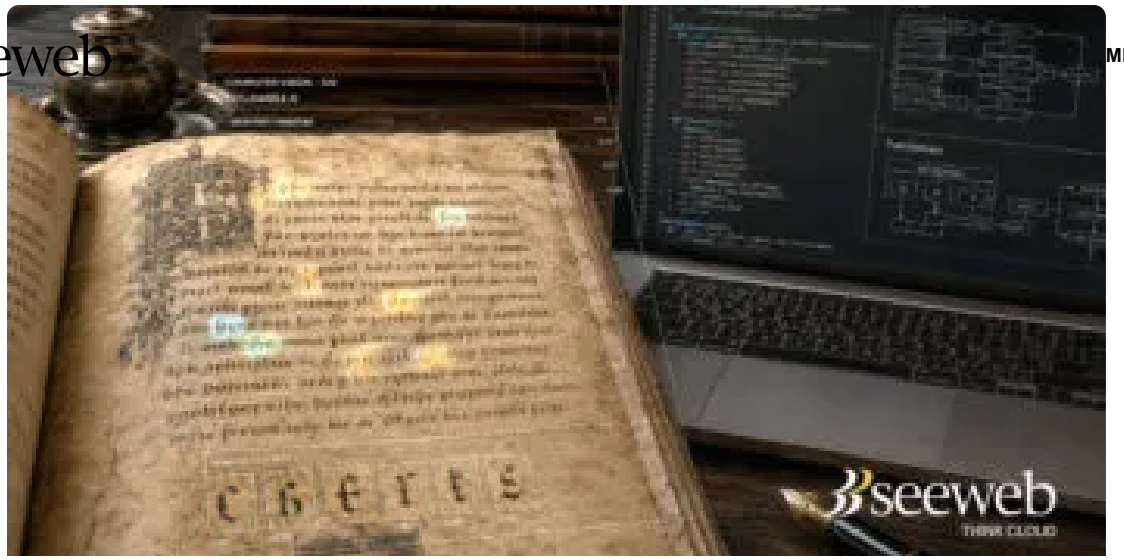
1. **Guide: Deploying LLMs with KubeAI on k3s (Ubuntu) (https://blog.seeweb.it/guide-deploying-llms-with-kubeai-on-k3s-ubuntu/)**
2. **Come addestrare un algoritmo di Machine Learning con tensorflow e keras: un caso d'uso di Cloud Server GPU (https://blog.seeweb.it/addestrare-algoritmo-di-machine-learning-con-tensorflow-e-keras/)**
3. **Getting Started with Ollama: A Hands-on Guide (https://blog.seeweb.it/getting-started-with-ollama-a-hands-on-guide/)**
4. **Machine Learning vs Deep Learning: approcci all'apprendimento e differenze (https://blog.seeweb.it/machine-learning-vs-deep-learning-approcci-allapprendimento-e-differenze/)**
5. **Accelerating LLM Inference with vLLM: A Hands-on Guide (https://blog.seeweb.it/accelerating-llm-inference-with-vllm-a-hands-on-guide/)**
6. **GPU Memory Slicing: Running Multiple AI Models with NVIDIA MIG (https://blog.seeweb.it/gpu-memory-slicing-running-multiple-ai-models-with-nvidia-mig/)**

Altri articoli da non perdere

(https://blog.seeweb.it/nextcloud-su-data-center-italiani-collaborazione-sicura-dati-in-italia-e-continuita-oper

Nextcloud su data center italiani: collaborazione sicura, dati in Italia e continuità operativa (https://blog.seeweb.it/nextcloud-su-data-center-italiani-collaborazione-sicura-dati-in-italia-e-continuita-operativa/)

28 Maggio 2026 • Nessun commento



(https://blog.seeweb.it/intelligenza-artificiale-e-manoscritti-medievali-quando-il-deep-learning-incontra-la-pa a/)

Intelligenza Artificiale e manoscritti medievali: quando il Deep Learning incontra la paleografia (https://blog.seeweb.it/intelligenza-artificiale-e-manoscritti-medievali-quando-il-deep-learning-incontra-la-paleografia/)

21 Maggio 2026 • Nessun commento

(https://blog.seeweb.it/reti-fibra-ottica-e-il-futuro-della-connettivita-in-italia-connesi-del-gruppo-dhh-punta -gbit-s/)

Reti, fibra ottica e il futuro della connettività in Italia: Connesi, del Gruppo DHH, punta a 400 Gbit/s (https://blog.seeweb.it/reti-fibra-ottica-e-il-futuro-della-connettivita-in-italia-connesi-del-gruppo-dhh-punta-a-400-gbit-s/)

30 Marzo 2026 • Nessun commento

Lascia un commento

Il tuo indirizzo email non sarà pubblicato. I campi obbligatori sono contrassegnati *

Commento *

Nome *

Email *

Sito web

Salva il mio nome, email e sito web in questo browser per la prossima volta che commento.

Dimostra di non essere un bot 60 − = 51

Invia commento

Email

info@seeweb.it (mailto:info@seeweb.it)

SEDI

Milano
Via Caldera 21
Blue Building, ala 1
20153 - Italia
(+39) 02 87365100

Frosinone
Via A. Vona 66
03100 Italia
(+39) 0775 880 041

Lugano
Via Vergiò - Breganzona
6932 Svizzera

Zurigo
Hofwisenstrasse 56 - Rümlang
8153 Svizzera

Sofia
16V Barzaritsa Str.
Ovcha Kupel District
1618 Bulgaria

seeweb

MENU